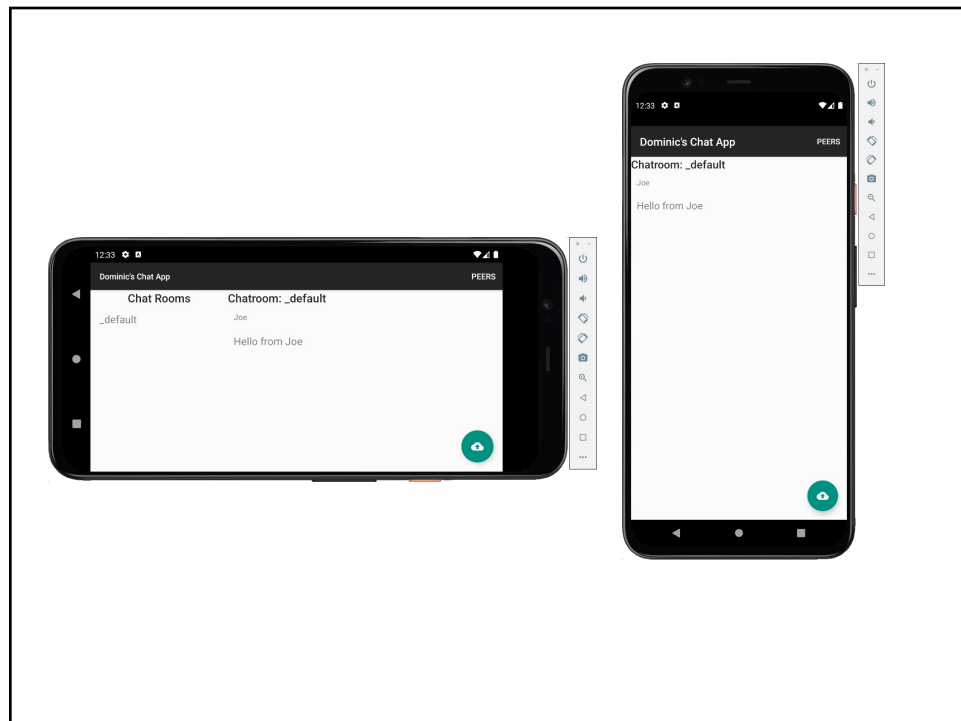


EXAMPLE: CHAT FRAGMENT

1



2

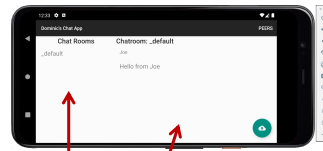
Landscape Layout

```
<LinearLayout xmlns:android=...
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <fragment android:name="...ChatroomsFragment"
        android:id="@+id/chatrooms_fragment"
        android:layout_weight="1"
        android:layout_width="0dp"
        android:layout_height="match_parent" />

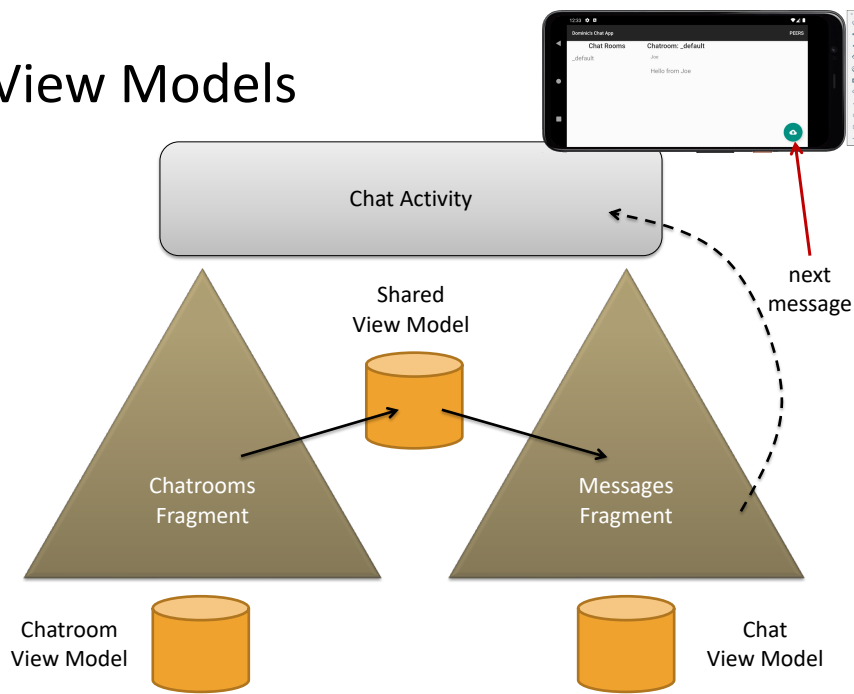
    <fragment android:name="...MessagesFragment"
        android:id="@+id/messages_fragment"
        android:layout_weight="2"
        android:layout_width="0dp"
        android:layout_height="match_parent" />

</LinearLayout>
```



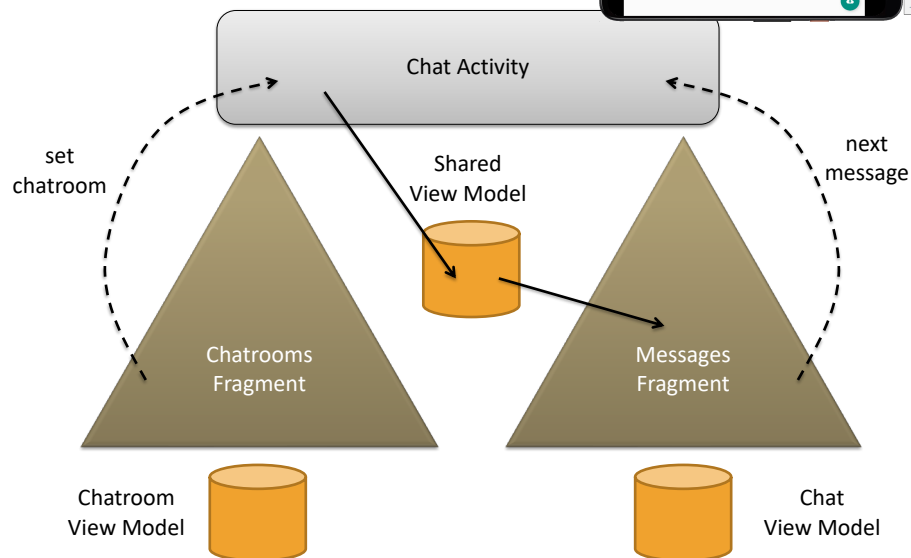
3

View Models



4

View Models



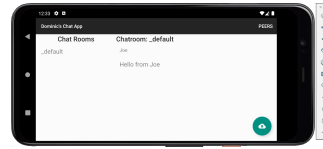
5

Chatrooms Fragment

```
public class ChatroomsFragment extends Fragment {  
    IChatroomListener listener;  
  
    public void onAttach(Context context) {  
        listener = (IChatRoomListener) context;  
    }  
  
    public void onItemClick(...) {  
        ...  
        listener.setChatroom(chatroom);  
    }  
}
```

6

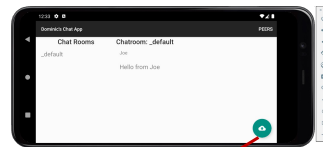
Chatrooms Fragment



```
public class ChatroomsFragment extends Fragment {  
  
    IChatRoomListener listener;  
  
    public void onCreateView(...) {  
        RecyclerView chatroomList = ...;  
        chatroomList.setAdapter(...);  
    }  
  
    public void onViewCreated(...) {  
        ...  
        ChatroomViewModel chatroomViewModel = ...;  
        ... chatroomViewModel.fetchAllChatrooms(); ...  
    }  
}
```

7

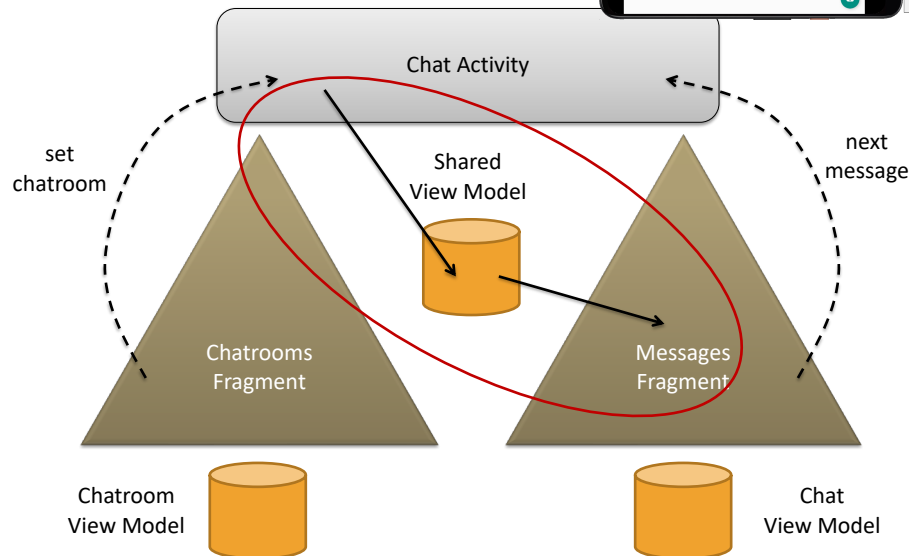
Messages Fragment



```
public class MessagesFragment extends Fragment {  
  
    IChatListener listener;  
  
    public void onAttach(Context context) {  
        listener = (IChatListener) context;  
    }  
  
    public void onClick(...) {  
        ...  
        listener.nextMessage();  
    }  
}
```

8

View Models



9

Shared View Model

```
public class SharedViewModel extends AndroidViewModel {
    private final MutableLiveData<Chatroom> selected =
        new MutableLiveData<>();

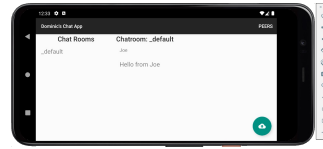
    public void select(Chatroom chatroom) {
        selected.setValue(chatroom);
    }

    public Chatroom getSelected() {
        return selected.getValue();
    }

    public void observe(..., Observer<Chatroom> observer) {
        selected.observe(ui, observer);
    }
}
```

10

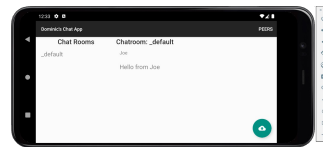
Messages Fragment



```
public class MessagesFragment extends Fragment {  
  
    SharedViewModel sharedViewModel;  
  
    public void onCreateView(...) {  
        RecyclerView messageList = ...;  
        messageList.setAdapter(...);  
    }  
  
    public void onViewCreated(...) {  
        ...  
        sharedViewModel = ...;  
        queryMessages(sharedViewModel.getSelected());  
        sharedViewModel.observe(..., this::queryMessages);  
    }  
}
```

11

Chat Activity

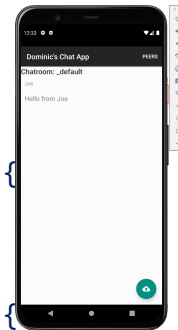


```
public class ChatServerActivity extends ... {  
  
    SharedViewModel sharedViewModel;  
  
    public void setChatroom(Chatroom chatroom) {  
        sharedViewModel.select(chatroom);  
    }  
  
}
```

12

Chat Activity

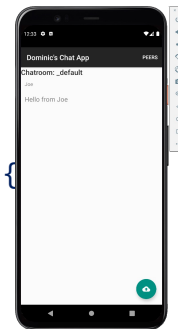
```
public class ChatServerActivity extends ... {  
  
    SharedViewModel sharedViewModel;  
  
    public void setChatroom(Chatroom chatrom) {  
        sharedViewModel.select(chatroom);  
        // Do this if NOT two pane  
        getSupportFragmentManager()  
            .beginTransaction()  
            .replace(..., new MessagesFragment())  
            .addToBackStack(...)  
            .commit();  
    }  
}
```



13

Chat Activity

```
public class ChatServerActivity extends ... {  
  
    public void onCreate(Chatroom chatrom) {  
        setContentView(...);  
        // Do this if NOT two pane  
        getSupportFragmentManager()  
            .beginTransaction()  
            .add(..., new ChatroomsFragment(), ...)  
            // Don't do this: .addToBackStack(...)  
            .commit();  
    }  
}
```

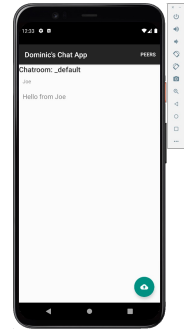


14

Portrait Layout

```
<FrameLayout xmlns:android=...  
    android:id="@+id/fragment_container"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent">
```

```
</FrameLayout>
```

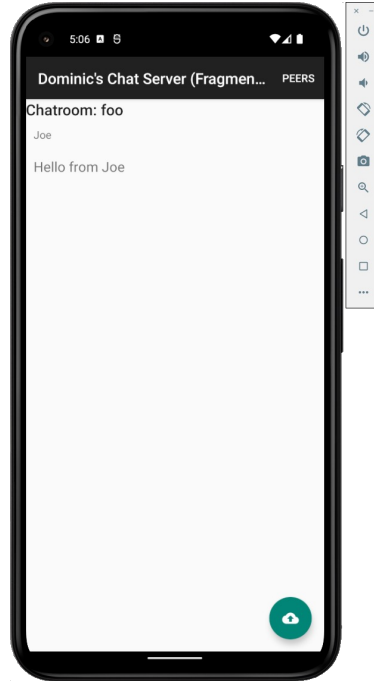


15

HANDLING BACK

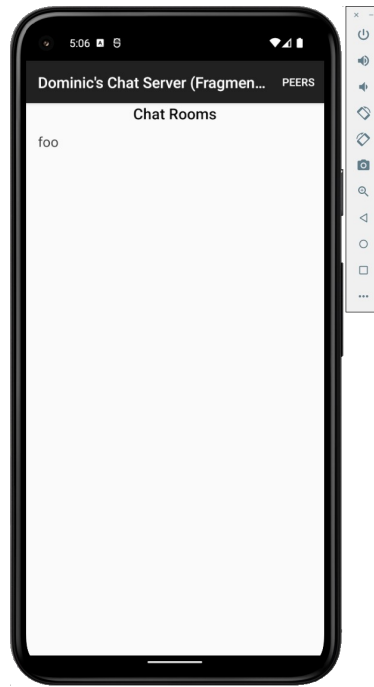
16

Handling Back



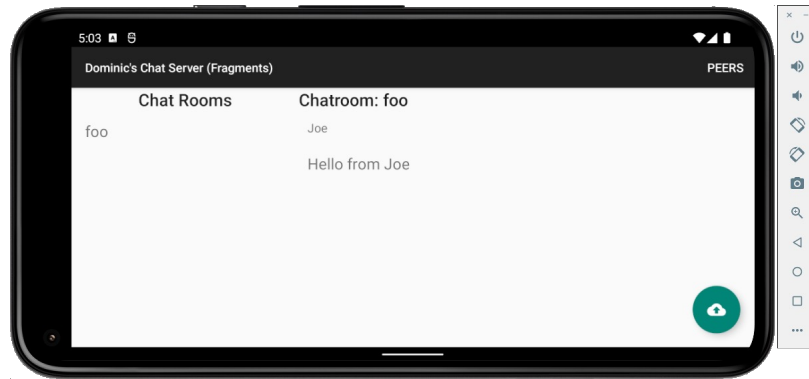
17

Handling Back



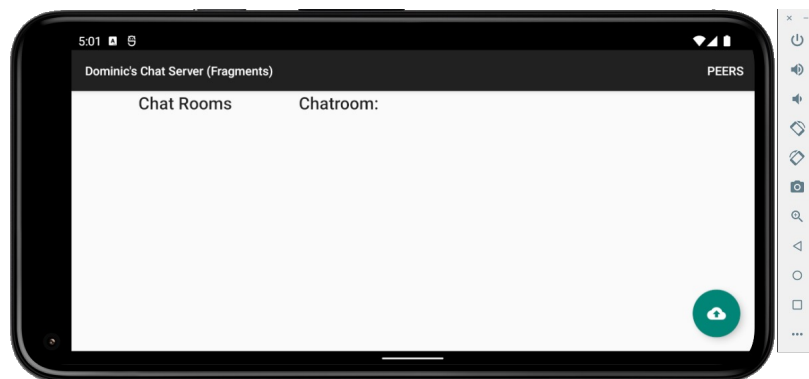
18

Handling Back



19

Handling Back



20

Handling Back

```
public class SharedViewModel extends AndroidViewModel {  
    public SharedViewModel(Application context) {  
        super(context);  
        selected = new MutableLiveData<>();  
    }  
    private final MutableLiveData<Chatroom> selected;  
    public void select(Chatroom chatroom) {  
        selected.setValue(chatroom);  
    }  
    public Chatroom getSelected() {  
        return selected.getValue();  
    }  
    public void observe(LifecycleOwner ui, Observer<Chatroom> observer) {  
        selected.observe(ui, observer);  
    }  
}
```

21

Handling Back

```
public class ChatServerActivity extends AppCompatActivity {  
    @Override  
    public void onBackPressed() {  
        if (!isTwoPane) {  
            super.onBackPressed();  
            return;  
        }  
        /*  
        * We are in two-pane mode, is a chatroom selected?  
        */  
        if (sharedViewModel.getSelected() == null) {  
            super.onBackPressed();  
            return;  
        }  
        /*  
        * Unset the currently selected chatroom (in two-pane mode).  
        */  
        setChatroom(null);  
    }  
}
```

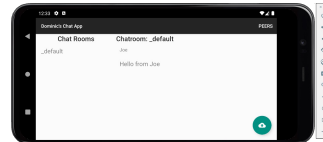
22

Handling Back

```
public class ChatServerActivity extends AppCompatActivity ... {  
  
    private OnBackPressedCallback callback;  
  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
  
        // TODO get the view models  
        ...  
        if (...in two-pane mode...) {  
            // In two-pane mode, prevent exiting app when chat room open.  
            callback = new OnBackPressedCallback(...) {  
                @Override  
                public void handleOnBackPressed() {  
                    // how to handle back press?  
                }  
            };  
            ...  
        }  
    }  
}
```

23

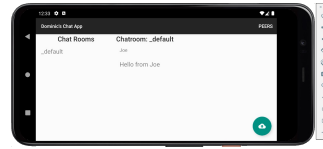
Handling Back



```
public class ChatServerActivity extends ... {  
  
    SharedViewModel sharedViewModel;  
  
    public void setChatroom(Chatroom chatroom) {  
        sharedViewModel.select(chatroom);  
        if (...single-pane mode...) {  
            // Replace ChatroomsFragment with MessagesFragment  
        }  
    }  
}
```

24

Handling Back



```
public class ChatServerActivity extends ... {  
  
    SharedViewModel sharedViewModel;  
  
    OnBackPressedCallback callback;  
  
    public void setChatroom(Chatroom chatroom) {  
        sharedViewModel.select(chatroom);  
        if (...single-pane mode...) {  
            // Replace ChatroomsFragment with MessagesFragment  
        } else {  
            // enable backpress callback if not null chatroom  
        }  
    }  
}
```